

String and Text Manipulation - Interview Questions

Solutions with code, output and explanation

Interview Questions

1. Remove Spaces from Given Text

Code:

```
def remove_spaces(text):  
    return text.replace(" ", "")  
  
print(remove_spaces("he llo wor ld"))
```

Output:

```
helloworld
```

Explanation: replace(' ', '') replaces every space with an empty string, so all spaces are removed.

2. Reverse a String

Code:

```
def reverse_string(text):  
    return text[::-1]  
  
print(reverse_string("hello"))
```

Output:

```
olleh
```

Explanation: Slicing with [::-1] reads the string from the end to the start, so it comes out reversed.

3. Reverse a String After Removing Spaces

Code:

```
def reverse_without_spaces(text):  
    text = text.replace(" ", "")  
    return text[::-1]  
  
print(reverse_without_spaces("he llo world"))
```

Output:

```
dlrowolleh
```

Explanation: First remove the spaces using replace(), then reverse the result with [::-1].

4. Convert Snake Case to Camel Case

Code:

```
def snake_to_camel(text):
    words = text.split("_")
    result = words[0]
    for word in words[1:]:
        result = result + word.capitalize()
    return result

print(snake_to_camel("my_variable_name"))
```

Output:

```
myVariableName
```

Explanation: Split at '_'. The first word stays as it is, and every other word gets a capital first letter using `capitalize()`.

5. Convert Snake Case to Pascal Case

Code:

```
def snake_to_pascal(text):
    words = text.split("_")
    result = ""
    for word in words:
        result = result + word.capitalize()
    return result

print(snake_to_pascal("my_variable_name"))
```

Output:

```
MyVariableName
```

Explanation: Same as `camelCase`, but here even the first word is capitalized.

6. Convert Camel Case to Snake Case

Code:

```
def camel_to_snake(text):
    result = ""
    for ch in text:
        if ch.isupper():
            result = result + "_" + ch.lower()
        else:
            result = result + ch
    return result

print(camel_to_snake("myVariableName"))
```

Output:

```
my_variable_name
```

Explanation: Loop through each character. If it is uppercase, add '_' before it and make it lowercase. Otherwise add it as it is.

7. Convert Camel Case to Pascal Case

Code:

```
def camel_to_pascal(text):  
    return text[0].upper() + text[1:]  
  
print(camel_to_pascal("myVariable"))
```

Output:

```
MyVariable
```

Explanation: Only the first letter needs to change. `text[0].upper()` capitalizes it and `text[1:]` is the remaining string.

8. Convert Pascal Case to Camel Case

Code:

```
def pascal_to_camel(text):  
    return text[0].lower() + text[1:]  
  
print(pascal_to_camel("MyVariable"))
```

Output:

```
myVariable
```

Explanation: Make only the first letter lowercase and keep the rest unchanged.

9. Convert Pascal Case to Snake Case

Code:

```
def pascal_to_snake(text):  
    result = ""  
    for i in range(len(text)):  
        ch = text[i]  
        if ch.isupper() and i != 0:  
            result = result + "_" + ch.lower()  
        else:  
            result = result + ch.lower()  
    return result  
  
print(pascal_to_snake("MyVariable"))
```

Output:

```
my_variable
```

Explanation: Same idea as camel to snake, but the first letter is also capital. So '_' is not added for index 0, and all letters are converted to lowercase.

10. Convert Text to Camel Case

Code:

```
def text_to_camel(text):
    words = text.split()
    result = words[0].lower()
    for word in words[1:]:
        result = result + word.capitalize()
    return result

print(text_to_camel("hello world example"))
```

Output:

```
helloWorldExample
```

Explanation: split() gives the words. The first word is lowercase and the other words are capitalized, then all are joined together.

11. Convert Text to Snake Case

Code:

```
def text_to_snake(text):
    return "_".join(text.lower().split())

print(text_to_snake("hello world example"))
```

Output:

```
hello_world_example
```

Explanation: Split the sentence into words and join them with '_'. lower() makes sure everything is lowercase.

12. Convert Text to Pascal Case

Code:

```
def text_to_pascal(text):
    words = text.split()
    result = ""
    for word in words:
        result = result + word.capitalize()
    return result

print(text_to_pascal("hello world example"))
```

Output:

```
HelloWorldExample
```

Explanation: Every word gets a capital first letter and the words are joined without spaces.

13. Swap Upper and Lower Case

Code:

```
def swap_case(text):
    return text.swapcase()

print(swap_case("HeLLo"))
```

Output:

```
hEllO
```

Explanation: swapcase() changes uppercase letters to lowercase and lowercase letters to uppercase.

14. Separate Digits from Text

Code:

```
def extract_digits(text):
    digits = ""
    for ch in text:
        if ch.isdigit():
            digits = digits + ch
    return digits

print(extract_digits("abc123d4"))
```

Output:

```
1234
```

Explanation: Check each character with `isdigit()` and collect only the digits in a new string.

15. Print Uppercase, Lowercase, Digits, and Special Characters Separately

Code:

```
text = "Abc123!@#"
upper = []
lower = []
digits = []
special = []

for ch in text:
    if ch.isupper():
        upper.append(ch)
    elif ch.islower():
        lower.append(ch)
    elif ch.isdigit():
        digits.append(ch)
    else:
        special.append(ch)

print("Uppercase:", " ".join(upper))
print("Lowercase:", " ".join(lower))
print("Digits:", " ".join(digits))
print("Special Characters:", " ".join(special))
```

Output:

```
Uppercase: A
Lowercase: b c
Digits: 1 2 3
Special Characters: ! @ #
```

Explanation: Each character is put into one of four lists using `isupper()`, `islower()` and `isdigit()`. Anything left is a special character. `join()` prints them separated by spaces.

16. Count of Uppercase, Lowercase, Digits, and Special Characters

Code:

```
text = "AbC@123x!"
upper = 0
lower = 0
digits = 0
special = 0

for ch in text:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
    elif ch.isdigit():
        digits += 1
    else:
        special += 1

print("Uppercase:", upper)
print("Lowercase:", lower)
print("Digits:", digits)
print("Special Characters:", special)
```

Output:

```
Uppercase: 2
Lowercase: 2
Digits: 3
Special Characters: 2
```

Explanation: Same logic as the previous question, but counters are used instead of lists. Note: the sample output in the question says Lowercase: 1, but the input "AbC@123x!" has two lowercase letters ('b' and 'x'), so the correct answer is 2.

17. Check Password Strength

Code:

```
def check_password(password):
    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False

    for ch in password:
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        else:
            has_special = True

    if has_upper and has_lower and has_digit and has_special:
        return "Strong Password"
    else:
        return "Weak Password"

print(check_password("Pass123!"))
print(check_password("pass123"))
```

Output:

```
Strong Password
Weak Password
```

Explanation: Four flags track whether the password has an uppercase letter, lowercase letter, digit and special character. It is strong only if all four are True. A second example shows a weak password.

18. Remove Duplicates in a Given Input

Code:

```
def remove_duplicates(text):
    result = ""
    for ch in text:
        if ch not in result:
            result = result + ch
    return result

print(remove_duplicates("aabbcc"))
```

Output:

```
abc
```

Explanation: A character is added to the result only if it is not already in the result, so each character appears once and the order is kept.

19. Print Duplicates in a Given String

Code:

```
def print_duplicates(text):
    duplicates = []
    for ch in text:
        if text.count(ch) > 1 and ch not in duplicates:
            duplicates.append(ch)
    print(" ".join(duplicates))

print_duplicates("aabbccde")
```

Output:

```
a b c
```

Explanation: If count() of a character is more than 1, it is a duplicate. The 'not in duplicates' check makes sure it is printed only once.

20. Print Next Characters in a Given String

Code:

```
def next_characters(text):
    result = ""
    for ch in text:
        result = result + chr(ord(ch) + 1)
    return result

print(next_characters("abc"))
```

Output:

```
bcd
```

Explanation: ord() gives the ASCII number of a character. Adding 1 and converting back with chr() gives the next character. (For 'z' this gives '{', since no wrap-around was asked.)